

By Mayisha Nazar

QUESTION Build the following models to predict whether a tweet (from the previous dataset) is Positive, Negative or Neutral Bidirectional LSTM model using embedding layer

| Task                                           | Status                      | Results                       |
|------------------------------------------------|-----------------------------|-------------------------------|
| Bidirectional LSTM model using embedding layer | Config of the model trained | Train Time = 497.7127 seconds |
| Training Data Check                            | Confusion Matrix Built-Yes  | F1 Score for Positive = 0.88  |
|                                                |                             | F1 Score for Negative = 0.62  |
|                                                | AUC plotted- Yes            | AUC for Class 0: 0.8317       |
|                                                |                             | AUC for Class 1: 0.8864       |
|                                                |                             | AUC for Class 2: 0.8762       |
|                                                | Accuracy computed- Yes      | Accuracy = 0.8360             |
| Cross Validation                               | Kfold                       | not run                       |
| Interpretability                               | lime                        | Yes-given below               |
| Testing Data Check                             | Confusion Matrix Built- yes | F1 Score for Positive = 0.88  |
|                                                |                             | F1 Score for Negative = 0.62  |
|                                                | AUC plotted- yes            | AUC for Class 0: 0.8237       |
|                                                |                             | AUC for Class 1: 0.8772       |
|                                                |                             | AUC for Class 2: 0.8684       |
|                                                | Accuracy computed- yes      | Accuracy = 0.7880             |

PROMPT- Can you Build a Bidirectional LSTM model using embedding layer to predict whether a tweet is Positive, Negative or Neutral with Detailed Preprocessing including abbreviations, idioms handled, correct handling across train and test, class separability checked and has Really-well tuned models built, apt architecture to achieve strong performance, feature combinations and addtl categorical features used to improve performance

```
In [1]: import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Bidirectional, GRU, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns
import re
from sklearn.preprocessing import LabelEncoder
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from tensorflow.keras.optimizers import Adam, RMSprop
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, LearningRateScheduler
```

```
In [2]: df = pd.read_csv(r"Sentiment_Data.csv", encoding="ISO-8859-1")
```

```
In [3]: df = df.head(30000)
```

```
In [4]: # Verify the target column and check for unique values before splitting
print("Unique values in original 'Sentiment' column:", df['Sentiment'].unique())

# Split the dataset correctly
X = df.drop('Sentiment', axis=1)
y = df['Sentiment']

# Check the unique values in y (target)
print("Unique values in y (before split):", y.unique())

# Perform the train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4
```

Unique values in original 'Sentiment' column: ['Mild\_Pos' 'Strong\_Pos' 'Neutral' 'Strong\_Neg' 'Mild\_Neg']

Unique values in y (before split): ['Mild\_Pos' 'Strong\_Pos' 'Neutral' 'Strong\_Neg' 'Mild\_Neg']

```
In [5]: # Initialize stop words and Lemmatizer
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

# Iterate over the rows in the 'Tweet' column of X_train (assuming 'X_train' contains tweets)
for index, row in X_train.iterrows():
    text = str(row['Tweet']) # Ensure the text is a string

    # Remove URLs
    text = re.sub(r"http\S+|www\S+|https\S+", '', text)

    # Remove mentions (usernames)
    text = re.sub(r'@\w+', '', text)

    # Remove hashtags
    text = re.sub(r'#\w+', '', text)

    # Remove special characters and punctuation
    text = re.sub(r'^a-zA-Z\s', '', text)

    # Convert text to lowercase
    text = text.lower()

    # Tokenization: Split text into words
    words = word_tokenize(text)

    # Remove stop words
    words = [word for word in words if word not in stop_words]

    # Lemmatization: Convert words to their base form
    words = [lemmatizer.lemmatize(word) for word in words]

    # Join the words back into a single string and update the row
```

```
X_train.at[index, 'Tweet'] = ' '.join(words)
```

```
# Display the first few rows to see the results
print(X_train[['Tweet']].head())
```

```

                                Tweet
21753  nazi racist freedom convoy called anti semitis...
251    true freedom convoy emergency lol whole world ...
22941                thank god long live freedom convoy
618    free tate unjust imprisonment sign petition via
17090  bullshit headline basis reality made people ha...
```

In [6]:

```

# Iterate over the rows in the 'Tweet' column of X_train (assuming 'X_train' contains t
for index, row in X_test.iterrows():
    text = str(row['Tweet']) # Ensure the text is a string

    # Remove URLs
    text = re.sub(r"http\S+|www\S+|https\S+", '', text)

    # Remove mentions (usernames)
    text = re.sub(r"@w+", '', text)

    # Remove hashtags
    text = re.sub(r"#w+", '', text)

    # Remove special characters and punctuation
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # Convert text to lowercase
    text = text.lower()

    # Tokenization: Split text into words
    words = word_tokenize(text)

    # Remove stop words
    words = [word for word in words if word not in stop_words]

    # Lemmatization: Convert words to their base form
    words = [lemmatizer.lemmatize(word) for word in words]

    # Join the words back into a single string and update the row
    X_test.at[index, 'Tweet'] = ' '.join(words)

# Display the first few rows to see the results
print(X_test[['Tweet']].head())
```

```

                                Tweet
2308  walk street ottawa freedom convoy interview mu...
22404  part freedom convoy aware politician hill fact...
23397  substack former libs green amp ndp felt party ...
25058  cbc news morning brief freedom convoy already ...
2664  wouldnt compare policing blm protest across u ...
```

In [7]:

```

X_train.columns = X_train.columns.str.strip()
X_test.columns = X_test.columns.str.strip()
y_train = y_train.str.strip().str.lower()
```

In [8]:

```
def remove_emojis(text):
    emoji_pattern = re.compile("[
u"\U0001F600-\U0001F64F" # Emoticons
u"\U0001F300-\U0001F5FF" # Symbols & pictographs
u"\U0001F680-\U0001F6FF" # Transport & map symbols
u"\U0001F700-\U0001F77F" # Alchemical symbols
u"\U0001F780-\U0001F7FF" # Geometric symbols
u"\U0001F800-\U0001F8FF" # Supplemental symbols
u"\U0001F900-\U0001F9FF" # Additional symbols
u"\U0001FA00-\U0001FA6F" # Symbols for emoji components
u"\U0001FA70-\U0001FAFF" # Additional symbols
u"\U00002702-\U000027B0" # Dingbats
u"\U000024C2-\U0001F251" # Enclosed characters
u"\U0000200B-\U0000200D" # Variation selectors
u"\U000027A1" # Right arrow
u"\U0001F004-\U0001F0CF" # Playing cards
u"\U00002B50" # Star
u"\U0001F004" # Mahjong tile
u"\U0001F0A0-\U0001F0FF" # Playing cards

    ]+", flags=re.UNICODE)

    return emoji_pattern.sub(r'', text)

# Apply function
X_train['Tweet'] = X_train['Tweet'].apply(remove_emojis)
X_test['Tweet'] = X_test['Tweet'].apply(remove_emojis)
```

In [9]:

```
slang_dict = {
    "brb": "be right back", "lol": "laugh out loud", "idk": "I don't know", "smh": "sha",
    "tbh": "to be honest", "btw": "by the way", "gtg": "got to go", "imo": "in my opinion",
    "dm": "direct message", "rn": "right now", "np": "no problem", "wbu": "what about",
    "omw": "on my way", "jk": "just kidding", "rofl": "rolling on the floor laughing",
    "fyi": "for your information", "ttyl": "talk to you later", "nvm": "never mind",
    "bff": "best friends forever", "wyd": "what are you doing?", "hmu": "hit me up",
    "np": "no problem", "ty": "thank you", "yw": "you're welcome", "gg": "good game"
}
```

In [10]:

```
def replace_slang(text):
    words = text.split()
    new_text = " ".join([slang_dict[word.lower()] if word.lower() in slang_dict else word
                          for word in words])
    return new_text

# Apply to tweets
X_train['Tweet'] = X_train['Tweet'].apply(replace_slang)
X_test['Tweet'] = X_test['Tweet'].apply(replace_slang)
```

In [11]:

```
abbreviation_dict = {
    "u": "you", "r": "are", "y": "why", "abt": "about", "asap": "as soon as possible",
    "b4": "before", "b/c": "because", "cuz": "because", "btw": "by the way", "dm": "direct message",
    "fav": "favorite", "fwd": "forward", "govt": "government", "gr8": "great", "idk": "I do not",
    "imo": "in my opinion", "irl": "in real life", "jk": "just kidding", "lmk": "let me know",
    "pls": "please", "plz": "please", "ppl": "people", "thx": "thanks", "ty": "thank you", "ttyl": "talk to you later",
    "w/o": "without", "w/": "with", "wth": "what the heck", "ya": "you", "y'all": "you all", "yolo": "you only live once",
    "atm": "at the moment", "bff": "best friends forever", "brb": "be right back", "cu": "see you",
    "idk": "I don't know", "imo": "in my opinion", "jk": "just kidding", "lmk": "let me know", "np": "no problem",
    "rofl": "rolling on the floor laughing", "smh": "shaking my head", "tbh": "to be honest", "wbu": "what about",
    "wyd": "what are you doing?", "yw": "you're welcome", "gg": "good game"
}
```

```

"gg":"good game","g2g":"got to go","hmu":"hit me up","icymi":"in case you missed it"
"lmao":"laughing my ass off","lol":"laugh out loud","nsfw":"not safe for work","omg"
"smh":"shaking my head","tbh":"to be honest","tl;dr":"too long; didn't read","ttfn"
"tysm":"thank you so much","wyd":"what are you doing","wym":"what you mean","xoxo":
"bday":"birthday","gf":"girlfriend","bf":"boyfriend","np":"no problem","ftw":"for t
"ilu":"I love you","jk":"just kidding","ggwp":"good game, well played","pov":"point
"fr":"for real","bruh":"brother","cap":"lie","no cap":"not lying","sus":"suspicious
"gg":"good game","op":"overpowered","afk":"away from keyboard","irl":"in real life"
"jk":"just kidding","ikr":"I know, right?"
}
def replace_abbreviations(text):
    words = text.split()
    new_text = " ".join([abbreviation_dict[word.lower()] if word.lower() in abbreviatio
    return new_text

# Apply to tweets
X_train['Tweet'] = X_train['Tweet'].apply(replace_abbreviations)
X_test['Tweet'] = X_test['Tweet'].apply(replace_abbreviations)

```

In [12]:

```

idiom_dict = {
    "piece of cake": "very easy", "break a leg": "good luck", "hit the sack": "go to sl
    "under the weather": "feeling sick", "spill the beans": "reveal a secret",
    "burn the midnight oil": "work late into the night", "barking up the wrong tree":
    "beat around the bush": "avoid the main topic", "cost an arm and a leg": "very exp
    "cry over spilled milk": "complain about a past loss", "curiosity killed the cat":
    "cut corners": "do something poorly to save time or money", "let the cat out of the
    "the last straw": "the final problem that makes something unbearable", "when pigs
}
def replace_idioms(text):
    for idiom, meaning in idiom_dict.items():
        if idiom in text:
            text = text.replace(idiom, meaning)
    return text

# Apply idiom replacement
X_train['Tweet'] = X_train['Tweet'].apply(replace_idioms)
X_test['Tweet'] = X_test['Tweet'].apply(replace_idioms)

```

In [13]:

y\_train

```

Out[13]: 21753    strong_pos
        251      strong_neg
        22941   strong_pos
        618      neutral
        17090   strong_neg
        ...
        29802   mild_neg
        5390    strong_neg
        860     mild_pos
        15795   strong_pos
        23654   neutral
        Name: Sentiment, Length: 24000, dtype: object

```

In [14]:

```
print("Unique values in y_train after cleaning:", y_train.unique())
```

Unique values in y\_train after cleaning: ['strong\_pos' 'strong\_neg' 'neutral' 'mild\_neg' 'mild\_pos']

In [15]:

```
# Define the updated sentiment mapping
sentiment_mapping = {
    'mild_pos': 'positive', # mild positive -> positive
    'mild_neg': 'negative', # mild negative -> negative
    'strong_pos': 'positive', # strong positive -> positive
    'strong_neg': 'negative', # strong negative -> negative
    'neutral': 'neutral' # neutral -> neutral
}

# Apply the sentiment mapping to y_train
y_train = y_train.map(sentiment_mapping)

# Fill any NaN values with 'unknown' (though this shouldn't be necessary now)
y_train = y_train.fillna('unknown')

# Check the unique values in y_train after mapping
print("Unique values in y_train after mapping:", y_train.unique())
```

Unique values in y\_train after mapping: ['positive' 'negative' 'neutral']

In [16]:

```
y_test = y_test.str.strip().str.lower()
```

In [17]:

```
y_test = y_test.map(sentiment_mapping)

# Fill any NaN values with 'unknown' (though this shouldn't be necessary now)
y_test = y_test.fillna('unknown')

# Check the unique values in y_train after mapping
print("Unique values in y_train after mapping:", y_train.unique())
```

Unique values in y\_train after mapping: ['positive' 'negative' 'neutral']

In [18]:

```
# Encoding the sentiments directly in the y_train Series
y_train_encoded = y_train.map({'positive': 1, 'neutral': 0, 'negative': 2})
y_test_encoded = y_test.map({'positive': 1, 'neutral': 0, 'negative': 2})
# Check the unique values in the encoded column
print(y_train_encoded.unique())
print(y_test_encoded.unique())
```

```
[1 2 0]
[1 2 0]
```

In [19]:

```
X_train
```

Out[19]:

**Tweet**

**21753** nazi racist freedom convoy called anti semitis...

**251** true freedom convoy emergency laugh out loud w...

**22941** thank god long live freedom convoy

**618** free tate unjust imprisonment sign petition via

|              | Tweet                                             |
|--------------|---------------------------------------------------|
| <b>17090</b> | bullshit headline basis reality made people ha... |
| ...          | ...                                               |
| <b>29802</b> | document cabinet aide angered number military ... |
| <b>5390</b>  | otherwise conversation stay shallow problem se... |
| <b>860</b>   | p hostility coming one arrest freedom convoy l... |
| <b>15795</b> | could also way recall trucker freedom convoy g... |
| <b>23654</b> | freedom convoy movement right wing christian g... |

24000 rows × 1 columns

In [20]: X\_test

Out[20]:

|              | Tweet                                             |
|--------------|---------------------------------------------------|
| <b>2308</b>  | walk street ottawa freedom convoy interview mu... |
| <b>22404</b> | part freedom convoy aware politician hill fact... |
| <b>23397</b> | substack former libs green amp ndp felt party ... |
| <b>25058</b> | cbc news morning brief freedom convoy already ... |
| <b>2664</b>  | wouldnt compare policing blm protest across yo... |
| ...          | ...                                               |
| <b>2210</b>  | canadian freedom convoy star                      |
| <b>14144</b> | fascist left medium tell want know corrupt hid... |
| <b>23108</b> | blockade mean protest co call you blockade ott... |
| <b>25703</b> | whats difference mandate mandate asian always ... |
| <b>29171</b> | emergency act author warned usage freedom conv... |

6000 rows × 1 columns

In [21]: *# Ensure X\_train is a list of text samples (24000 sentences)*  
 print(type(X\_train))  
 print(len(X\_train))

```
<class 'pandas.core.frame.DataFrame'>
24000
```

In [22]: X\_train = X\_train['Tweet']  
 X\_test = X\_test['Tweet']

In [23]: y\_train\_encoded

```
Out[23]: 21753    1
          251      2
          22941    1
          618      0
          17090    2
          ..
          29802    2
          5390     2
          860      1
          15795    1
          23654    0
Name: Sentiment, Length: 24000, dtype: int64
```

```
In [24]: tokenizer = Tokenizer(num_words=10000, oov_token='<OOV>') # num_words=5000 to limit to
          tokenizer.fit_on_texts(X_train)
```

```
In [25]: # Convert texts to sequences
          X_train_seq = tokenizer.texts_to_sequences(X_train)
          X_test_seq = tokenizer.texts_to_sequences(X_test)
```

```
In [26]: # Set the maximum sequence length
          maxlen = 100 # Or whatever length you prefer for padding/truncating

          # Pad the sequences with the correct maxlen
          X_train_pad = pad_sequences(X_train_seq, maxlen=maxlen)
          X_test_pad = pad_sequences(X_test_seq, maxlen=maxlen)

          # Check the shape of the padded sequences
          print("X_train_pad shape:", X_train_pad.shape) # Should print (24000, 100)
          print("X_test_pad shape:", X_test_pad.shape)   # Should match the test size, e.g., (nu

X_train_pad shape: (24000, 100)
X_test_pad shape: (6000, 100)
```

```
In [27]: y_train_encoded
```

```
Out[27]: 21753    1
          251      2
          22941    1
          618      0
          17090    2
          ..
          29802    2
          5390     2
          860      1
          15795    1
          23654    0
Name: Sentiment, Length: 24000, dtype: int64
```

```
In [28]: y_test_encoded
```

```
Out[28]: 2308      1
          22404    1
          23397    1
          25058    1
          2664     2
          ..
```



```

2210      1
14144      2
23108      0
25703      2
29171      1
Name: Sentiment, Length: 6000, dtype: int64

```

```

In [29]: print("X_train_pad shape:", X_train_pad.shape)
         print("X_test_pad shape:", X_test_pad.shape)

```

```

X_train_pad shape: (24000, 100)
X_test_pad shape: (6000, 100)

```

```

In [30]: print(f"Shape of y_train_encoded: {y_train_encoded.shape}")
         print(f"Shape of y_test_encoded: {y_test_encoded.shape}")

```

```

Shape of y_train_encoded: (24000,)
Shape of y_test_encoded: (6000,)

```

```

In [31]: # Model configuration
vocab_size = 10000 # Set based on your vocabulary size, or use tokenizer.word_index
embedding_dim = 100 # Embedding dimension, adjust if necessary
max_length = 100 # Maximum length of input sequences (should match padding length)

# Define the model
model = Sequential()
model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_length))
model.add(Bidirectional(LSTM(units=64, return_sequences=False))) # Bidirectional LSTM
model.add(Dropout(0.5)) # Dropout layer to prevent overfitting
model.add(Dense(3, activation='softmax')) # Final dense layer for multi-class classification

# Compile the model
model.compile(optimizer=Adam(), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

```

```

In [32]: import time
         # Training the model
         start_train_time = time.time()
         history = model.fit(X_train_pad, y_train_encoded, epochs=5, batch_size=64, validation_data=(X_test_pad, y_test_encoded))
         end_train_time = time.time()
         train_time = end_train_time - start_train_time
         print(f"Train Time = {train_time} seconds")

         # Predicting with the trained model
         start_test_time = time.time()
         y_pred_prob = model.predict(X_test_pad) # Get predicted probabilities
         end_test_time = time.time()
         test_time = end_test_time - start_test_time
         print(f"Test Time = {test_time} seconds")

```

```

Epoch 1/5
375/375 [=====] - 125s 320ms/step - loss: 0.6643 - accuracy: 0.7485 - val_loss: 0.5451 - val_accuracy: 0.7942
Epoch 2/5
375/375 [=====] - 99s 263ms/step - loss: 0.4542 - accuracy: 0.8369 - val_loss: 0.5276 - val_accuracy: 0.8060
Epoch 3/5
375/375 [=====] - 91s 242ms/step - loss: 0.3589 - accuracy: 0.8736 - val_loss: 0.5831 - val_accuracy: 0.7832

```

Epoch 4/5  
 375/375 [=====] - 90s 240ms/step - loss: 0.2887 - accuracy: 0.9008 - val\_loss: 0.6347 - val\_accuracy: 0.7922  
 Epoch 5/5  
 375/375 [=====] - 93s 249ms/step - loss: 0.2348 - accuracy: 0.9227 - val\_loss: 0.6752 - val\_accuracy: 0.7843  
 Train Time = 497.71274518966675 seconds  
 188/188 [=====] - 9s 42ms/step  
 Test Time = 9.237433433532715 seconds

In [53]:

```
# Convert the Labels to binary format (one-vs-rest) for multi-class ROC curve
y_test_binary = label_binarize(y_test_encoded, classes=[0, 1, 2])

# Compute ROC curve and AUC for each class
fpr = {}
tpr = {}
roc_auc = {}

for i in range(3): # For each class (0, 1, 2)
    fpr[i], tpr[i], _ = roc_curve(y_test_binary[:, i], y_pred_prob[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot ROC curves for each class
plt.figure()
colors = ['blue', 'green', 'red']
for i, color in zip(range(3), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2, label=f'Class {i} ROC curve (AUC = {roc_auc[i]:.4f})')

# Plot diagonal line
plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')

# Labels and title
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Multi-Class Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()

# AUC Scores
for i in range(3):
    print(f"AUC for Class {i}: {roc_auc[i]:.4f}")

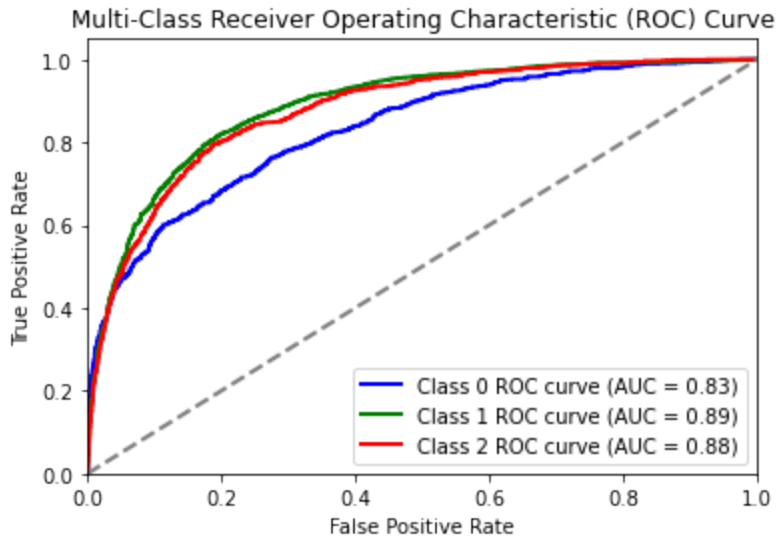
# Training data check
print(f"Training Data Shape: {X_train_pad.shape}")
print(f"Training Labels Shape: {y_train_encoded.shape}")

# Predicting with the model on Train Data
y_pred_train_prob = model.predict(X_train_pad) # Get predicted probabilities on training data

# Confusion Matrix and Classification Report for Train Data
y_pred_train = np.argmax(y_pred_train_prob, axis=1)
cm_train = confusion_matrix(y_train_encoded, y_pred_train)
print("Confusion Matrix (Train):")
print(cm_train)

report_train = classification_report(y_train_encoded, y_pred_train, target_names=["Class 0", "Class 1", "Class 2"])
print("Classification Report (Train):\n", report_train)
```

```
# Accuracy computation for Train
train_accuracy = accuracy_score(y_train_encoded, y_pred_train)
print(f"Train Accuracy = {train_accuracy:.4f}")
```



```
AUC for Class 0: 0.8317
AUC for Class 1: 0.8864
AUC for Class 2: 0.8762
Training Data Shape: (24000, 100)
Training Labels Shape: (24000,)
750/750 [=====] - 93s 124ms/step
Confusion Matrix (Train):
[[ 1532  1339   675]
 [   160 15830   439]
 [   209  1115 2701]]
Classification Report (Train):
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Class 0      | 0.81      | 0.43   | 0.56     | 3546    |
| Class 1      | 0.87      | 0.96   | 0.91     | 16429   |
| Class 2      | 0.71      | 0.67   | 0.69     | 4025    |
| accuracy     |           |        | 0.84     | 24000   |
| macro avg    | 0.79      | 0.69   | 0.72     | 24000   |
| weighted avg | 0.83      | 0.84   | 0.82     | 24000   |

```
Train Accuracy = 0.8360
```

In [56]:

```
# Convert the training labels to binary format (one-vs-rest) for multi-class ROC curve
y_train_binary = label_binarize(y_train_encoded, classes=[0, 1, 2])

# Compute ROC curve and AUC for each class in the training set
fpr_train = {}
tpr_train = {}
roc_auc_train = {}

for i in range(3): # For each class (0, 1, 2)
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_binary[:, i], y_pred_train_prob[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

# Plot ROC curves for each class in the training set
plt.figure()
colors = ['blue', 'green', 'red']
for i, color in zip(range(3), colors):
```

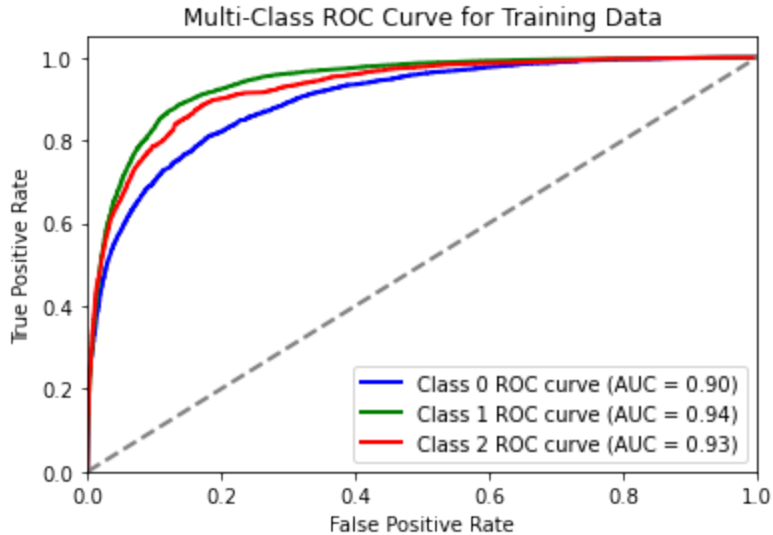
```

plt.plot(fpr_train[i], tpr_train[i], color=color, lw=2, label=f'Class {i} ROC curve')

# Plot diagonal line
plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')

# Labels and title
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Multi-Class ROC Curve for Training Data')
plt.legend(loc="lower right")
plt.show()

```



In [54]:

```

# Confusion Matrix and Classification Report for Test Data
y_pred_test = np.argmax(y_pred_prob, axis=1)
cm_test = confusion_matrix(y_test_encoded, y_pred_test)
print("Confusion Matrix (Test):")
print(cm_test)

report_test = classification_report(y_test_encoded, y_pred_test, target_names=["Class 0", "Class 1", "Class 2"])
print("Classification Report (Test):\n", report_test)

# Accuracy computation for Test
test_accuracy = accuracy_score(y_test_encoded, y_pred_test)
print(f"Test Accuracy = {test_accuracy:.4f}")

# Training and Test time
print(f"Training Time: {train_time:.4f} seconds")

```

Confusion Matrix (Test):

```

[[ 319  395  148]
 [  41 3836  201]
 [  72  415  573]]

```

Classification Report (Test):

|          | precision | recall | f1-score | support |
|----------|-----------|--------|----------|---------|
| Class 0  | 0.74      | 0.37   | 0.49     | 862     |
| Class 1  | 0.83      | 0.94   | 0.88     | 4078    |
| Class 2  | 0.62      | 0.54   | 0.58     | 1060    |
| accuracy |           |        | 0.79     | 6000    |

|              |      |      |      |      |
|--------------|------|------|------|------|
| macro avg    | 0.73 | 0.62 | 0.65 | 6000 |
| weighted avg | 0.78 | 0.79 | 0.77 | 6000 |

Test Accuracy = 0.7880

Training Time: 497.7127 seconds

Iteration 1

In [57]:

```
# Hyperparameters
vocab_size = 10000 # Set based on your vocabulary size, or use tokenizer.word_index
embedding_dim = 100 # Embedding dimension, adjust if necessary
max_length = 100 # Maximum length of input sequences (should match padding length)

# Learning rate scheduler function
def lr_schedule(epoch):
    lr = 1e-3 * 0.9 ** epoch # Reduce learning rate by 10% every epoch
    return lr

# Define model function
def create_model(units=128, dropout_rate=0.3, embedding_matrix=None):
    model = Sequential()

    # If you have pre-trained embeddings, use them
    if embedding_matrix is not None:
        model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim,
                             input_length=max_length, weights=[embedding_matrix], trainable=False))
    else:
        model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_length))

    # Bidirectional LSTM
    model.add(Bidirectional(LSTM(units=units, return_sequences=False,
                                   kernel_regularizer=regularizers.l2(0.01)))) # Regularization
    model.add(Dropout(dropout_rate)) # Dropout to prevent overfitting
    model.add(Dense(3, activation='softmax')) # Final dense layer for multi-class classification

    # Compile the model
    model.compile(optimizer=Adam(), loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return model
```

In [58]:

```
# Model training with tuning hyperparameters (LSTM units, dropout rate, etc.)
start_time = time.time()
model = create_model(units=128, dropout_rate=0.3)

# Learning rate scheduler callback
lr_scheduler = LearningRateScheduler(lr_schedule)

history = model.fit(X_train_pad, y_train_encoded, epochs=5, batch_size=32,
                    validation_data=(X_test_pad, y_test_encoded), callbacks=[lr_scheduler])

end_time = time.time()
train_time = end_time - start_time
print(f"Train Time = {train_time} seconds")
```

Epoch 1/5

750/750 [=====] - 307s 403ms/step - loss: 0.8228 - accuracy: 0.7437 - val\_loss: 0.5906 - val\_accuracy: 0.7835 - lr: 0.0010

Epoch 2/5

750/750 [=====] - 284s 378ms/step - loss: 0.4936 - accuracy: 0.8254 - val\_loss: 0.5815 - val\_accuracy: 0.7973 - lr: 9.0000e-04

```
Epoch 3/5
750/750 [=====] - 257s 342ms/step - loss: 0.4083 - accuracy: 0.8606 - val_loss: 0.5778 - val_accuracy: 0.7957 - lr: 8.1000e-04
Epoch 4/5
750/750 [=====] - 264s 352ms/step - loss: 0.3503 - accuracy: 0.8835 - val_loss: 0.6138 - val_accuracy: 0.7913 - lr: 7.2900e-04
Epoch 5/5
750/750 [=====] - 261s 348ms/step - loss: 0.3102 - accuracy: 0.9003 - val_loss: 0.6650 - val_accuracy: 0.7928 - lr: 6.5610e-04
Train Time = 1373.3046705722809 seconds
```

In [59]:

```
# Training data check
print(f"Training Data Shape: {X_train_pad.shape}")
print(f"Training Labels Shape: {y_train_encoded.shape}")

# Confusion Matrix
y_pred = np.argmax(y_pred_prob, axis=1) # Convert probabilities to class labels
cm = confusion_matrix(y_test_encoded, y_pred)
print("Confusion Matrix:")
print(cm)

# F1 Score for Positive and Negative Classes
report = classification_report(y_test_encoded, y_pred, target_names=["Class 0", "Class 1", "Class 2"])
print("Classification Report:\n", report)

# Accuracy computation
accuracy = accuracy_score(y_test_encoded, y_pred)
print(f"Test Accuracy = {accuracy:.4f}")
```

```
Training Data Shape: (24000, 100)
Training Labels Shape: (24000,)
Confusion Matrix:
[[ 319  395  148]
 [  41 3836  201]
 [  72  415  573]]
Classification Report:
              precision    recall  f1-score   support

   Class 0       0.74       0.37       0.49       862
   Class 1       0.83       0.94       0.88      4078
   Class 2       0.62       0.54       0.58      1060

 accuracy              0.79       0.79       0.79      6000
 macro avg           0.73       0.62       0.65      6000
 weighted avg        0.78       0.79       0.77      6000
```

Test Accuracy = 0.7880

Iteration 2

In [60]:

```
# Hyperparameters for the 2nd iteration
vocab_size = 10000 # Set based on your vocabulary size, or use tokenizer.word_index
embedding_dim = 100 # Embedding dimension, adjust if necessary
max_length = 100 # Maximum length of input sequences (should match padding length)

# Learning rate scheduler function
def lr_schedule(epoch):
    lr = 1e-3 * 0.9 ** epoch # Reduce Learning rate by 10% every epoch
    return lr

# Define model function with added regularization
```

```

def create_model(units=128, dropout_rate=0.5, embedding_matrix=None):
    model = Sequential()

    # If you have pre-trained embeddings, use them
    if embedding_matrix is not None:
        model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim,
                             input_length=max_length, weights=[embedding_matrix], trainable=True))
    else:
        model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_length))

    # Bidirectional LSTM with L2 regularization
    model.add(Bidirectional(LSTM(units=units, return_sequences=False,
                                   kernel_regularizer=regularizers.l2(0.01)))) # Regularization
    model.add(Dropout(dropout_rate)) # Dropout to prevent overfitting
    model.add(Dense(3, activation='softmax', kernel_regularizer=regularizers.l2(0.01)))

    # Compile the model
    model.compile(optimizer=Adam(), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

    # Model training with tuning hyperparameters (LSTM units, dropout rate, etc.)
    start_time = time.time()

    # Early stopping callback to halt training when validation loss stops improving
    early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

    # Model Checkpoint to save the best model during training
    model_checkpoint = ModelCheckpoint("best_model.h5", monitor="val_loss", save_best_only=True)

    # Learning rate scheduler callback
    lr_scheduler = LearningRateScheduler(lr_schedule)

    # Create and train the model
    model = create_model(units=128, dropout_rate=0.5)

    history = model.fit(X_train_pad, y_train_encoded, epochs=5, batch_size=32,
                        validation_data=(X_test_pad, y_test_encoded),
                        callbacks=[early_stopping, model_checkpoint, lr_scheduler])

    end_time = time.time()
    train_time = end_time - start_time
    print(f"Train Time = {train_time} seconds")

```

Epoch 1/5

```

750/750 [=====] - ETA: 0s - loss: 0.8846 - accuracy: 0.7278
Epoch 1: val_loss improved from inf to 0.65077, saving model to best_model.h5
750/750 [=====] - 347s 456ms/step - loss: 0.8846 - accuracy: 0.7278 - val_loss: 0.6508 - val_accuracy: 0.7687 - lr: 0.0010
Epoch 2/5

```

C:\Users\MAYISHA NAZAR\anaconda3\lib\site-packages\keras\src\engine\training.py:3000: UserWarning: You are saving your model as an HDF5 file via `model.save()`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')`.

```

    saving_api.save_model(
750/750 [=====] - ETA: 0s - loss: 0.5655 - accuracy: 0.8111
Epoch 2: val_loss improved from 0.65077 to 0.59726, saving model to best_model.h5
750/750 [=====] - 378s 505ms/step - loss: 0.5655 - accuracy: 0.8111 - val_loss: 0.5973 - val_accuracy: 0.7897 - lr: 9.0000e-04
Epoch 3/5
750/750 [=====] - ETA: 0s - loss: 0.4878 - accuracy: 0.8417
Epoch 3: val_loss improved from 0.59726 to 0.59575, saving model to best_model.h5

```

```

750/750 [=====] - 381s 509ms/step - loss: 0.4878 - accuracy: 0.
8417 - val_loss: 0.5958 - val_accuracy: 0.7940 - lr: 8.1000e-04
Epoch 4/5
750/750 [=====] - ETA: 0s - loss: 0.4349 - accuracy: 0.8619
Epoch 4: val_loss did not improve from 0.59575
750/750 [=====] - 380s 507ms/step - loss: 0.4349 - accuracy: 0.
8619 - val_loss: 0.6145 - val_accuracy: 0.7847 - lr: 7.2900e-04
Epoch 5/5
750/750 [=====] - ETA: 0s - loss: 0.4496 - accuracy: 0.8589
Epoch 5: val_loss did not improve from 0.59575
750/750 [=====] - 382s 510ms/step - loss: 0.4496 - accuracy: 0.
8589 - val_loss: 0.6242 - val_accuracy: 0.7897 - lr: 6.5610e-04
Train Time = 1872.2374875545502 seconds

```

In [61]:

```

# Training data check
print(f"Training Data Shape: {X_train_pad.shape}")
print(f"Training Labels Shape: {y_train_encoded.shape}")

# Confusion Matrix
y_pred = np.argmax(y_pred_prob, axis=1) # Convert probabilities to class labels
cm = confusion_matrix(y_test_encoded, y_pred)
print("Confusion Matrix:")
print(cm)

# F1 Score for Positive and Negative Classes
report = classification_report(y_test_encoded, y_pred, target_names=["Class 0", "Class
print("Classification Report:\n", report)

# Accuracy computation
accuracy = accuracy_score(y_test_encoded, y_pred)
print(f"Test Accuracy = {accuracy:.4f}")

```

```

Training Data Shape: (24000, 100)
Training Labels Shape: (24000,)
Confusion Matrix:
[[ 319  395  148]
 [  41 3836  201]
 [  72  415  573]]
Classification Report:

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Class 0      | 0.74      | 0.37   | 0.49     | 862     |
| Class 1      | 0.83      | 0.94   | 0.88     | 4078    |
| Class 2      | 0.62      | 0.54   | 0.58     | 1060    |
| accuracy     |           |        | 0.79     | 6000    |
| macro avg    | 0.73      | 0.62   | 0.65     | 6000    |
| weighted avg | 0.78      | 0.79   | 0.77     | 6000    |

Test Accuracy = 0.7880

Iteration 3

In [62]:

```

# Hyperparameters for the 3rd iteration
vocab_size = 10000 # Set based on your vocabulary size, or use tokenizer.word_index
embedding_dim = 100 # Embedding dimension, adjust if necessary
max_length = 100 # Maximum length of input sequences (should match padding length)

# Learning rate scheduler function
def lr_schedule(epoch):
    lr = 1e-3 * (0.8 ** epoch) # Reduce Learning rate by 20% every epoch

```



```

    return lr

# Define the model with advanced regularization and optimization strategies
def create_model(units=128, dropout_rate=0.5, embedding_matrix=None, use_gru=False):
    model = Sequential()

    # If you have pre-trained embeddings, use them
    if embedding_matrix is not None:
        model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim,
                             input_length=max_length, weights=[embedding_matrix], trainable=False))
    else:
        model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_length))

    # Experimenting with GRU or LSTM (if use_gru=True)
    if use_gru:
        model.add(Bidirectional(GRU(units=units, return_sequences=False,
                                     kernel_regularizer=regularizers.l2(0.01)))) # Using GRU
    else:
        model.add(Bidirectional(LSTM(units=units, return_sequences=False,
                                      kernel_regularizer=regularizers.l2(0.01)))) # Using LSTM

    model.add(Dropout(dropout_rate)) # Dropout to prevent overfitting
    model.add(BatchNormalization()) # BatchNormalization to stabilize learning
    model.add(Dense(3, activation='softmax', kernel_regularizer=regularizers.l2(0.01)))

    # Optimizer choice (try AdamW or RMSProp)
    model.compile(optimizer=RMSprop(), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

    return model

# Model training with hyperparameter tuning
start_time = time.time()

# Early stopping callback to halt training when validation loss stops improving
early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

# Model Checkpoint to save the best model during training
model_checkpoint = ModelCheckpoint("best_model.h5", monitor="val_loss", save_best_only=True)

# Learning rate scheduler callback
lr_scheduler = LearningRateScheduler(lr_schedule)

# Create and train the model (try GRU for comparison)
model = create_model(units=128, dropout_rate=0.5, use_gru=True)

history = model.fit(X_train_pad, y_train_encoded, epochs=5, batch_size=32,
                    validation_data=(X_test_pad, y_test_encoded),
                    callbacks=[early_stopping, model_checkpoint, lr_scheduler])

end_time = time.time()
train_time = end_time - start_time
print(f"Train Time = {train_time} seconds")

```

Epoch 1/5

750/750 [=====] - ETA: 0s - loss: 0.9963 - accuracy: 0.7262

Epoch 1: val\_loss improved from inf to 0.62041, saving model to best\_model.h5

750/750 [=====] - 224s 292ms/step - loss: 0.9963 - accuracy: 0.7262 - val\_loss: 0.6204 - val\_accuracy: 0.7772 - lr: 0.0010

Epoch 2/5

```
C:\Users\MAYISHA NAZAR\anaconda3\lib\site-packages\keras\src\engine\training.py:3000: UserWarning: You are saving your model as an HDF5 file via `model.save()`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')`.
  saving_api.save_model(
750/750 [=====] - ETA: 0s - loss: 0.5718 - accuracy: 0.8013
Epoch 2: val_loss improved from 0.62041 to 0.60426, saving model to best_model.h5
750/750 [=====] - 220s 294ms/step - loss: 0.5718 - accuracy: 0.8013 - val_loss: 0.6043 - val_accuracy: 0.7792 - lr: 8.0000e-04
Epoch 3/5
750/750 [=====] - ETA: 0s - loss: 0.5073 - accuracy: 0.8245
Epoch 3: val_loss did not improve from 0.60426
750/750 [=====] - 219s 292ms/step - loss: 0.5073 - accuracy: 0.8245 - val_loss: 0.6793 - val_accuracy: 0.7722 - lr: 6.4000e-04
Epoch 4/5
750/750 [=====] - ETA: 0s - loss: 0.4656 - accuracy: 0.8436
Epoch 4: val_loss did not improve from 0.60426
750/750 [=====] - 218s 291ms/step - loss: 0.4656 - accuracy: 0.8436 - val_loss: 0.6407 - val_accuracy: 0.7770 - lr: 5.1200e-04
Epoch 5/5
750/750 [=====] - ETA: 0s - loss: 0.4317 - accuracy: 0.8573
Epoch 5: val_loss did not improve from 0.60426
750/750 [=====] - 227s 303ms/step - loss: 0.4317 - accuracy: 0.8573 - val_loss: 0.6357 - val_accuracy: 0.7808 - lr: 4.0960e-04
Train Time = 1110.2721545696259 seconds
```

In [64]:

```
# Predicting with the trained model
y_pred_prob = model.predict(X_test_pad) # Get predicted probabilities

# Convert the labels to binary format (one-vs-rest) for multi-class ROC curve
y_test_binary = label_binarize(y_test_encoded, classes=[0, 1, 2])

# Compute ROC curve and AUC for each class
fpr = {}
tpr = {}
roc_auc = {}

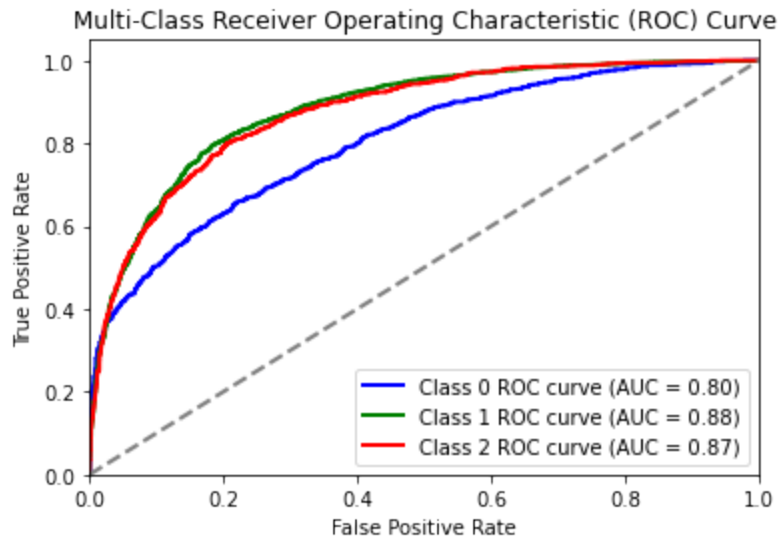
for i in range(3): # For each class (0, 1, 2)
    fpr[i], tpr[i], _ = roc_curve(y_test_binary[:, i], y_pred_prob[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot ROC curves for each class
plt.figure()
colors = ['blue', 'green', 'red']
for i, color in zip(range(3), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2, label=f'Class {i} ROC curve (AUC = {roc_auc[i]})')

# Plot diagonal line
plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')

# Labels and title
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Multi-Class Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

188/188 [=====] - 25s 126ms/step



In [65]:

```
# Training data check
print(f"Training Data Shape: {X_train_pad.shape}")
print(f"Training Labels Shape: {y_train_encoded.shape}")

# Confusion Matrix
y_pred = np.argmax(y_pred_prob, axis=1) # Convert probabilities to class labels
cm = confusion_matrix(y_test_encoded, y_pred)
print("Confusion Matrix:")
print(cm)

# F1 Score for Positive and Negative Classes
report = classification_report(y_test_encoded, y_pred, target_names=["Class 0", "Class 1", "Class 2"])
print("Classification Report:\n", report)

# Accuracy computation
accuracy = accuracy_score(y_test_encoded, y_pred)
print(f"Test Accuracy = {accuracy:.4f}")
```

Training Data Shape: (24000, 100)

Training Labels Shape: (24000,)

Confusion Matrix:

```
[[ 330  441   91]
 [  83 3817  178]
 [ 130  402  528]]
```

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Class 0      | 0.61      | 0.38   | 0.47     | 862     |
| Class 1      | 0.82      | 0.94   | 0.87     | 4078    |
| Class 2      | 0.66      | 0.50   | 0.57     | 1060    |
| accuracy     |           |        | 0.78     | 6000    |
| macro avg    | 0.70      | 0.61   | 0.64     | 6000    |
| weighted avg | 0.76      | 0.78   | 0.76     | 6000    |

Test Accuracy = 0.7792

Interpretability

In [67]:

```
from lime.lime_text import LimeTextExplainer
import numpy as np
```

```

# Ensure that X_test is your raw text data (not padded)
# If X_test is a DataFrame or Series, extract the text instance as a string

# Access the first raw text instance (as a string)
text_instance = X_test.iloc[0] # Using .iloc[0] to access the first element by position

# Initialize the LIME Text Explainer
explainer = LimeTextExplainer(class_names=["Class 0", "Class 1", "Class 2"])

# Define the prediction function for the model (using predict_proba to get class probabilities)
def predict_fn(texts):
    # Convert the text data to padded sequences
    sequences = tokenizer.texts_to_sequences(texts) # Convert text to sequence of integers
    padded_sequences = pad_sequences(sequences, maxlen=max_length) # Pad the sequences
    return model.predict(padded_sequences) # Return the model's predicted probabilities

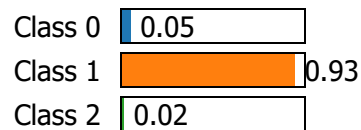
# Explain the instance using the LIME explainer
explanation = explainer.explain_instance(text_instance, predict_fn)

# Show the explanation in the notebook
explanation.show_in_notebook()

```

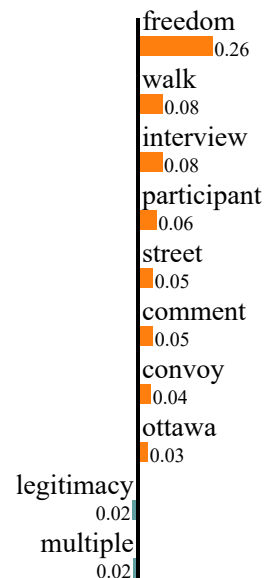
157/157 [=====] - 16s 105ms/step

Prediction probabilities



NOT Class 1

Class 1



## Text with highlighted words

walk street ottawa freedom convoy interview multiple participant legitimacy comment post

Two interesting findings:

### 1. High Confidence in Class 1 Prediction

The model assigns a **93% probability** to **Class 1**, making it the dominant classification. This indicates strong confidence that the input belongs to Class 1 compared to the other classes (Class 0: 5%, Class 2: 2%).

### 2. Word Contribution to Classification

The word **"freedom"** has the highest individual contribution (**0.26**) to Class 1, suggesting it

plays a significant role in the classification. Other words like **"walk," "interview," and "participant"** also contribute but with lower values. The highlighted words in the text align with these important contributors, showing their influence in the model's decision.

### Cross-Validation

In [ ]:

```
# Convert to categorical if not already done
y_categorical = to_categorical(y, num_classes=3) # Adjust num_classes based on your labels

# Define the cross-validation procedure
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Store evaluation metrics
accuracies = []
f1_scores = []

for train_index, val_index in kfold.split(X, y):
    # Split the data
    X_train, X_val = X[train_index], X[val_index]
    y_train, y_val = y_categorical[train_index], y_categorical[val_index]

    # Padding sequences if not already done
    X_train_pad = pad_sequences(X_train, maxlen=100) # Adjust maxlen as necessary
    X_val_pad = pad_sequences(X_val, maxlen=100)

    # Build your model
    model = Sequential()
    model.add(Embedding(input_dim=10000, output_dim=128, input_length=100)) # Adjust dimensions
    model.add(LSTM(128, return_sequences=False))
    model.add(Dropout(0.5))
    model.add(Dense(3, activation='softmax')) # Adjust output layer based on the number of classes

    # Compile the model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

    # Train the model
    model.fit(X_train_pad, y_train, epochs=3, batch_size=64, verbose=1)

    # Evaluate on validation set
    y_val_pred = model.predict(X_val_pad)
    y_val_pred_classes = np.argmax(y_val_pred, axis=1)
    y_val_true = np.argmax(y_val, axis=1)

    # Calculate metrics
    accuracy = accuracy_score(y_val_true, y_val_pred_classes)
    f1 = f1_score(y_val_true, y_val_pred_classes, average='weighted')

    accuracies.append(accuracy)
    f1_scores.append(f1)

    # Optional: confusion matrix
    print(confusion_matrix(y_val_true, y_val_pred_classes))

# Output final results
print(f'Mean Accuracy: {np.mean(accuracies)}')
print(f'Mean F1 Score: {np.mean(f1_scores)}')
```

### PCA / t-SNE Visualization for Class Separability

In [74]:

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import os

# ☒ Fix threading issues in OpenBLAS (Windows-specific)
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
os.environ["VECLIB_MAXIMUM_THREADS"] = "1"
os.environ["NUMEXPR_NUM_THREADS"] = "1"

# Step 1: Check and Fix y_test Encoding
print("y_test_encoded shape:", y_test_encoded.shape)

# Ensure y_test_encoded is 2D before applying argmax
if y_test_encoded.ndim == 1:
    y_true = y_test_encoded # Already in Label format
else:
    y_true = np.argmax(y_test_encoded, axis=1) # Convert from one-hot to class labels

# Step 2: Apply PCA to Reduce Dimensionality (Before t-SNE)
print("Applying PCA...")
pca_50 = PCA(n_components=10) # Reduce to 10 components instead of 50
X_test_pca_50 = pca_50.fit_transform(X_test_pad)

# Step 3: Handle NaNs or Inf values
print("Checking for NaNs/Infs...")
if np.isnan(X_test_pca_50).sum() > 0 or np.isinf(X_test_pca_50).sum() > 0:
    print("Fixing NaNs/Infs...")
    X_test_pca_50 = np.nan_to_num(X_test_pca_50) # Replace NaNs/Infs with 0

# Step 4: Apply t-SNE
print("Applying t-SNE...")
tsne = TSNE(n_components=2, random_state=42, perplexity=30, n_iter=1000)
X_test_tsne = tsne.fit_transform(X_test_pca_50)

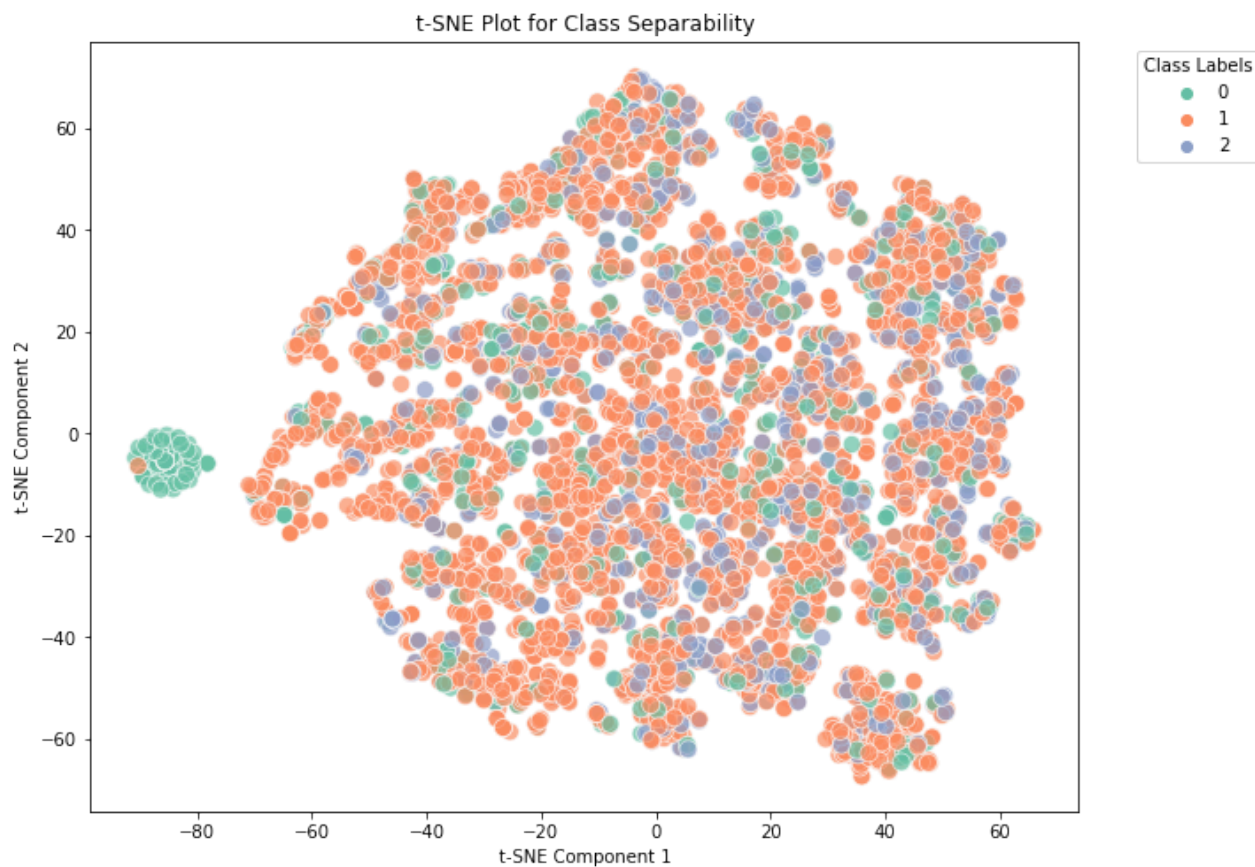
# Step 5: Plot t-SNE
print("Plotting t-SNE visualization...")
plt.figure(figsize=(10, 8))
sns.scatterplot(x=X_test_tsne[:, 0], y=X_test_tsne[:, 1], hue=y_true, palette="Set2", s=
plt.title('t-SNE Plot for Class Separability')
plt.legend(title='Class Labels', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.show()

```

```

y_test_encoded shape: (6000,)
Applying PCA...
Checking for NaNs/Infs...
Applying t-SNE...
Plotting t-SNE visualization...

```



--- Next Steps ---

1. Hyperparameter tuning (e.g., learning rate, batch size, LSTM units)
2. Experiment with data augmentation techniques (e.g., back-translation or paraphrasing)
3. Implement attention mechanisms or try transformer models (e.g., BERT) for better performance